Artificial Intelligence

A.Y. 2025/2026

Prof. Fabio Patrizi

Searching

Search Problems

Fundamental class of problems in AI

A *Search Problem* is defined by:

- *State space*: all possible world states (can be infinite)
- *Initial state*: in what state the world starts
- *Goal state(s)*: in what state(s) the agent would like to be
- *Actions*: what the agent can do in each state
- *Transition model*: how the state changes when an action is performed
- *Action-cost function*: represents cost of performing action in given state and ending up in successor state

Search Problems

$P=<S, A, s_0, G \subseteq S, \alpha: S \rightarrow \mathcal{P}(S), \delta: SxA \rightarrow S, c: SxAxS \rightarrow \mathbb{R}>$

- *S: State space*
- s_0*: Initial state*
- *G: Goal states*
- *A: Action space*
- α*: Action-precondition function*
- δ*: Transition function*
- *c: Action-cost function*

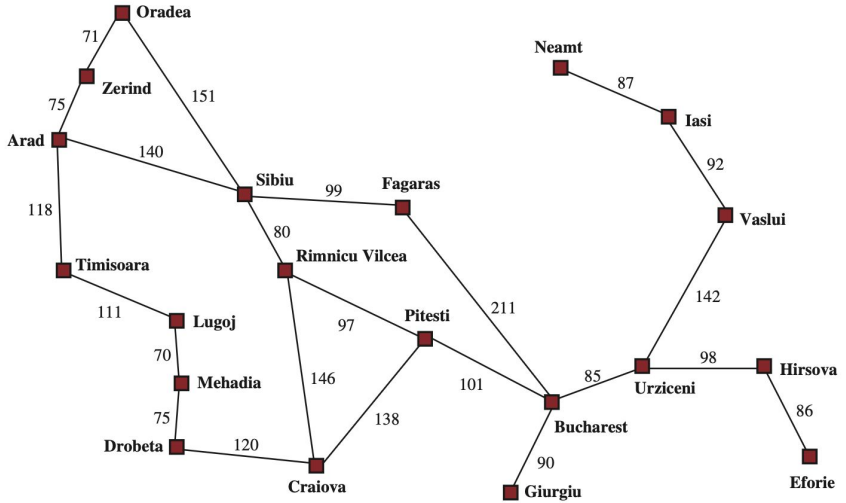
Example

Agent can move along edges

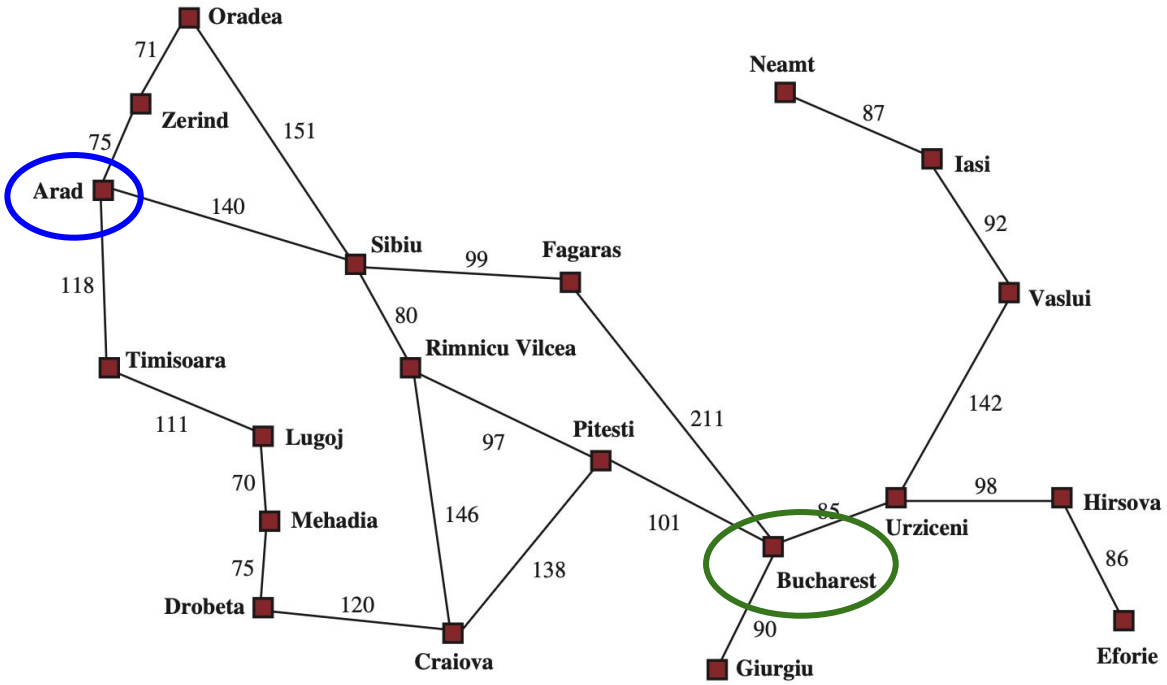
Edge weights model distances

Environment is (this is the simplest case):

- fully observable
- single agent
- deterministic
- static
- discrete

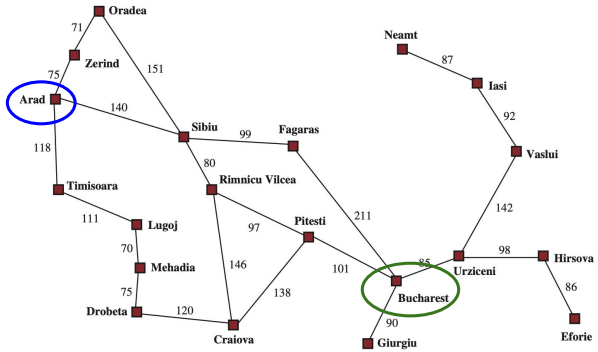


Example

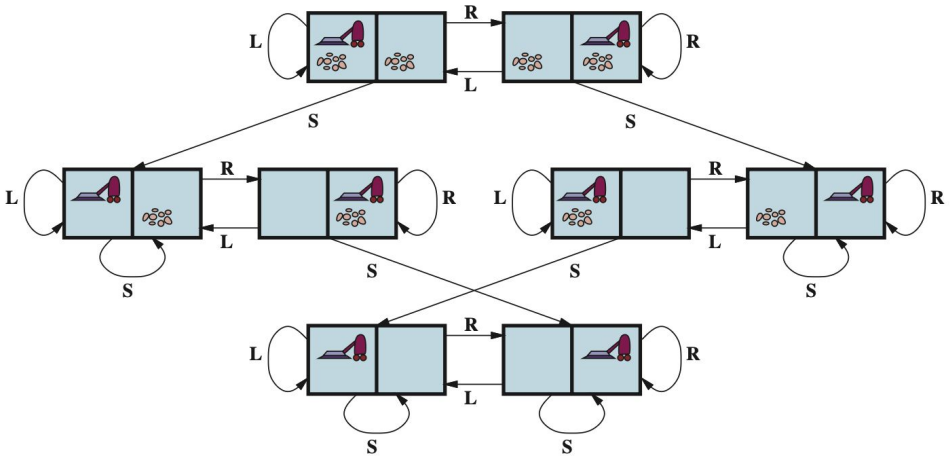

Oradea
71
Neamt
87
Zerind
151
Iasi
75
Arad
140
92
Sibiu
99
Fagaras
118
Vaslui
80
Rimnicu Vilcea
Timisoara
142
111
Pitesti
211
Lugoj
97
98
Hirsova
70
146
101
85
Mehadia
Urziceni
75
138
86
Bucharest
Drobeta
120
90
Craiova
Giurgiu
Eforie

Example

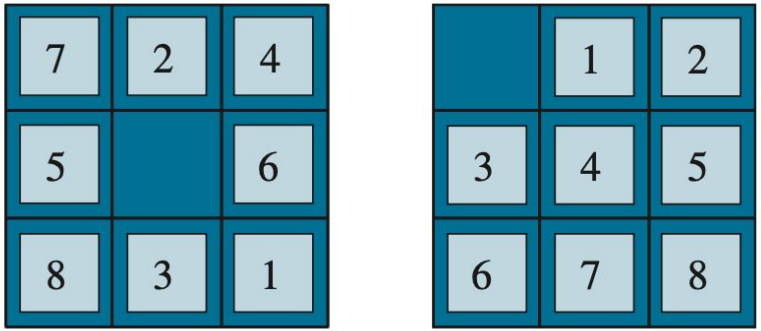
- State space: all possible agent's positions
  - E.g.: *Sibiu*
- Initial state: (agent in) *Arad*
- Goal state: *Bucharest*
- Actions: *applicable* moves along edges
  - E.g.: α(*Arad*)={To*Zerind, ToSibiu, ToTimisoara*}
- *Transition model*: all allowed moves
  - δ(*Arad, ToZerind*)=*Zerind*
- *Action-cost function*: road distance
  - E.g.: c(*Arad,ToSibiu,Sibiu*)=140
  - For deterministic environment: c(*Arad,ToSibiu*)=140 (successor state is known)



Example: vacuum world

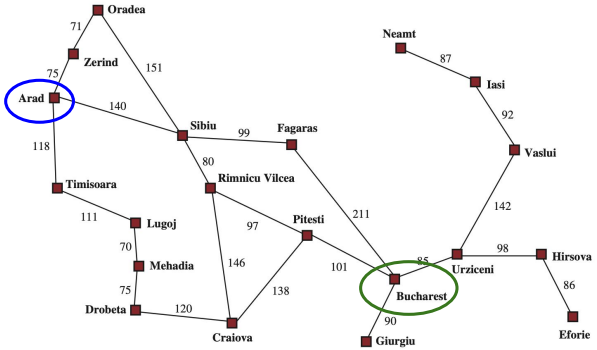


Example: 8-puzzle



Start State Goal State

Formalization



Useful notions:

- *Path*: sequence of actions
  - E.g.: *ToTimisoara, ToLugoj, ToMehadia*
- *Solution*: path leading from initial to (some) goal state (only for deterministic)
  - E.g.: *ToArad, ToSibiu, ToFagaras, ToBucharest* (cost: 450)
- *Optimal* solution: solution of minimal cost (sum of costs)
  - E.g.: *ToSibiu, ToRimnicuVilcea, ToPitesti, ToBucharest* (cost: 418)

Search Algorithm

Search Algorithm: algorithm that solves a Search Problem

Typically (but not always) uses *Search Tree*

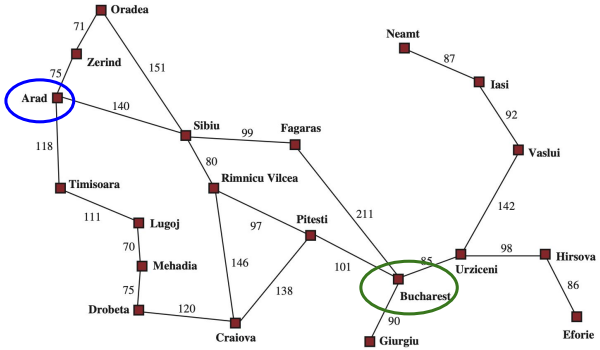
Search Tree (ST)

Fundamental notion in AI

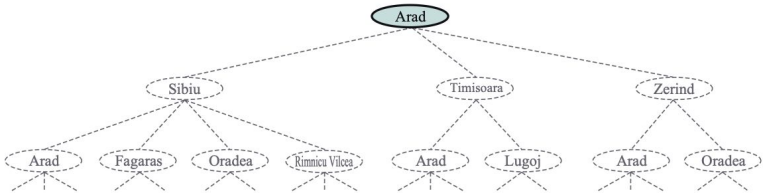
Tree defined by transition model, with:

- *Nodes* corresponding to states of the problem
- *Root* corresponding to initial problem's state
- *Edges* corresponding to actions

Search Tree



Transition Model



Search Tree

WARNING: node ≠ state

Paths in Search Tree

- ST Paths are action sequences applicable from initial state
- Tree Paths from initial state to some goal state are solutions
- We can solve problem by searching a path in the Search Tree

Observe: Search Tree can be infinite!

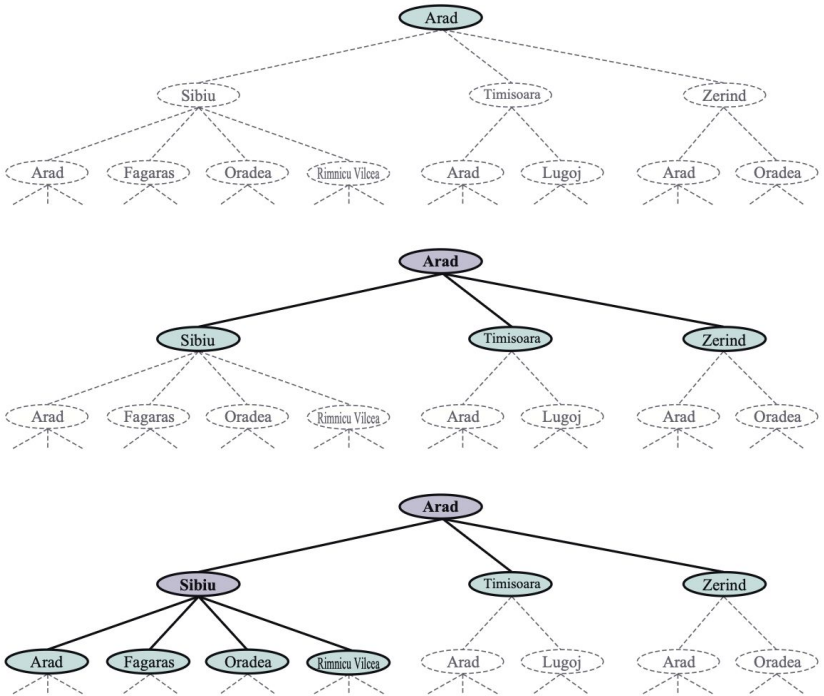
Node Expansion

Node expansion:

- Visit all applicable actions
- Basic search step on ST

Frontier:

- Set of unexpanded nodes



Node Expansion

Basic search algorithms are based on repeated node expansions

Until goal state reached

Key point: which node to expand next?

Loops, tree-like and graph-like search

In the presence of loops, we need to mark states already visited

We also need to record cost of getting to state, as searching for optimal solutions

Two search approaches:

- *Graph-like search*: avoid getting stuck in loops, by marking
- *Tree-like search: no marking, can get stuck*

Best-first Search

General approach to searching

- Assume *evaluation function f(n)* modeling how bad a state is wrt goal
- Expand node *n* with minimum value *f(n)* first (Best node first)
- Different evaluation functions yield different search approaches

Best-first Search

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

- *node*.STATE: the state to which the node corresponds;
- *node*.PARENT: the node in the tree that generated this node;
- *node*.ACTION: the action that was applied to the parent's state to generate this node;
- *node*.PATH-COST: the total cost of the path from the initial state to this node. In mathematical formulas, we use $g(node)$ as a synonym for PATH-COST.

We need a data structure to store the **frontier**. The appropriate choice is a **queue** of some kind, because the operations on a frontier are:

- IS-EMPTY(*frontier*) returns true only if there are no nodes in the frontier.
- POP(*frontier*) removes the top node from the frontier and returns it.
- TOP(*frontier*) returns (but does not remove) the top node of the frontier.
- ADD(*node*, *frontier*) inserts node into its proper place in the queue.

Properties of (Search) Algorithms

- Soundness:
  - When the algorithm returns non-failure, is that a solution?
- Completeness:
  - Does the algorithm find a solution when there is one?
- Optimality:
  - Is the returned solution minimal-cost?
- Complexity:
  - Time: How many states/actions are considered, depending on input size?
  - Space: How much memory is required?

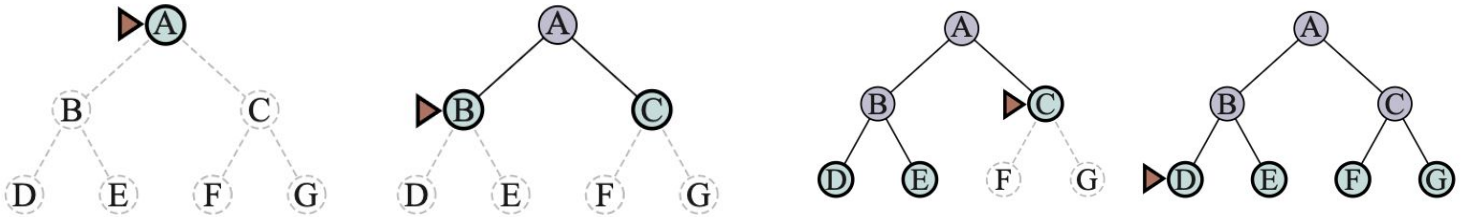
Uninformed Search

Uninformed Search: no information about distance to goal is used (or available)

- Different evaluation functions *f(n)* yield different search approaches
- Can still use current (root-to-node) cost to expand

Breadth-first Search

- Assume all actions have same cost (e.g.: 1)
- Choose *f(n)=depth-of-node-n* (shallower nodes are better)
- Best-first then reduces to Breadth First



Optimizations: use queue instead of *f(n)*; use set of *reached* states instead of table; *early goal test*

Sound, Complete, Optimal

Complexity: $O(b^d)$ for both time and space; b: branching factor; *d*: depth of shallower goal state

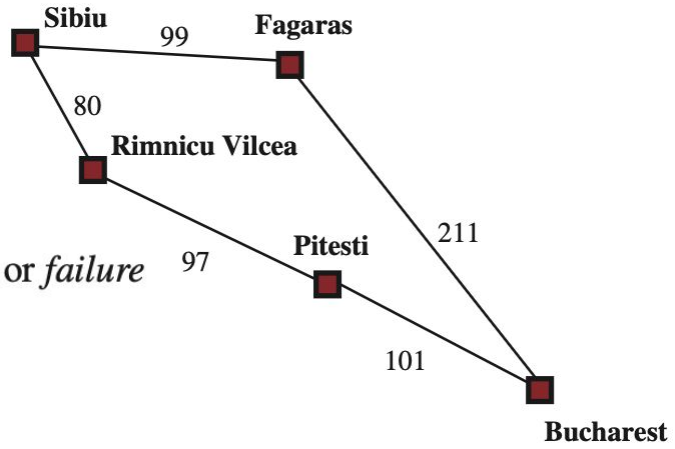
Breadth-first Search

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

Uniform-cost Search (a.k.a. Dijkstra's Algorithm)

Applicable if cost are strictly positive

f(n)=cost of root-to-n path (a.k.a. *path cost*)



function UNIFORM-COST-SEARCH(*problem*) **returns** a solution node, or *failure*
  return BEST-FIRST-SEARCH(*problem*, PATH-COST)

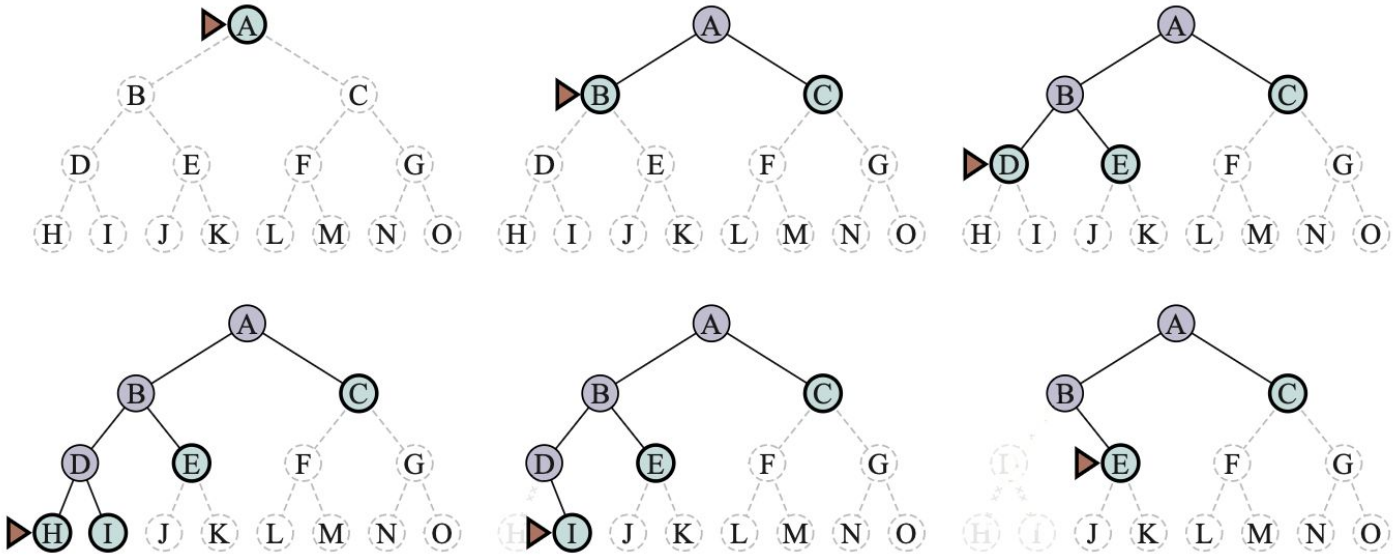
Sound, Complete, Optimal

Time and Space complexity: $O(b^{1+(C^*/\epsilon)})$

- *C*: cost of optimal solution*
- ϵ*: min cost of*

Depth-first Search

f(n)= -(depth-of-n)



Depth-first Search

- Not Optimal :(
- Sound
- Incomplete, in general
  - For infinite state-spaces can get stuck visiting some infinite path
- Complete over finite state spaces
  - may visit same node multiple times

Depth-first Search

However:

- Very convenient when tree-search like is feasible
  - Feasibility depends on structure of transition model, e.g., tree-shaped
- Memory efficient!
  - Stores in memory only root-to-node path
  - Space complexity: *O(db)*, with d depth of tree

Depth-first Search Variants

How to avoid getting stuck down infinite paths?

- Depth-limited Limit search up to some depth limit l
  - Time: $O(b^l)$
  - Space: $O(bl)$
- Iterative-deepening: Increase l iteratively
  - Time: $O(b^d)$
  - Space: $O(bd)$

May not find solution if one exists, even for finite state space

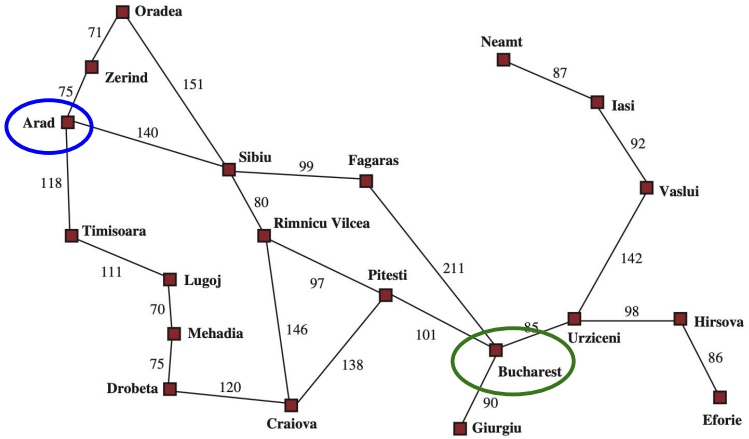
Uninformed Search: Summary

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]	Yes[1,4]
Optimal cost?	Yes[3]	Yes	No	No	Yes[3]	Yes[3,4]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figure 3.15 Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: [1] complete if b is finite, and the state space either has a solution or is finite. [2] complete if all action costs are $\geq \epsilon > 0$; [3] cost-optimal if action costs are all identical; [4] if both directions are breadth-first or uniform-cost.

Heuristic Search

Assume you know straight-line distance between Bucharest and other cities



Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

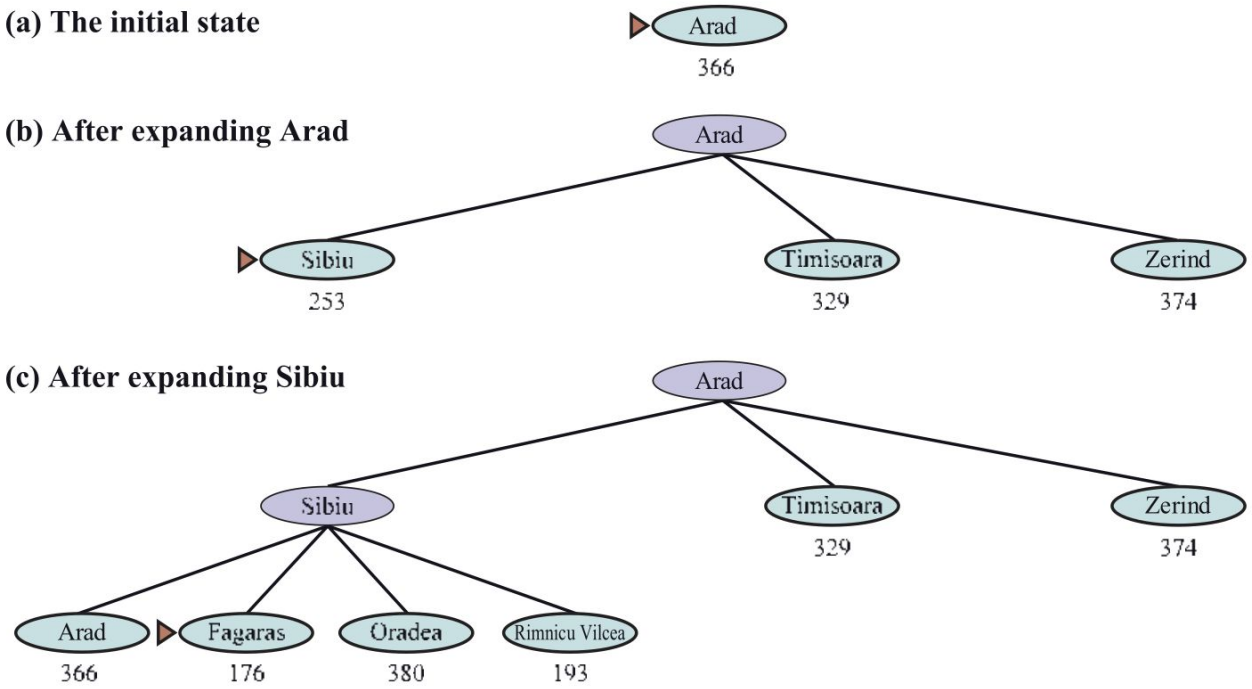
How can we use such information to improve search?

Greedy Best-first Search

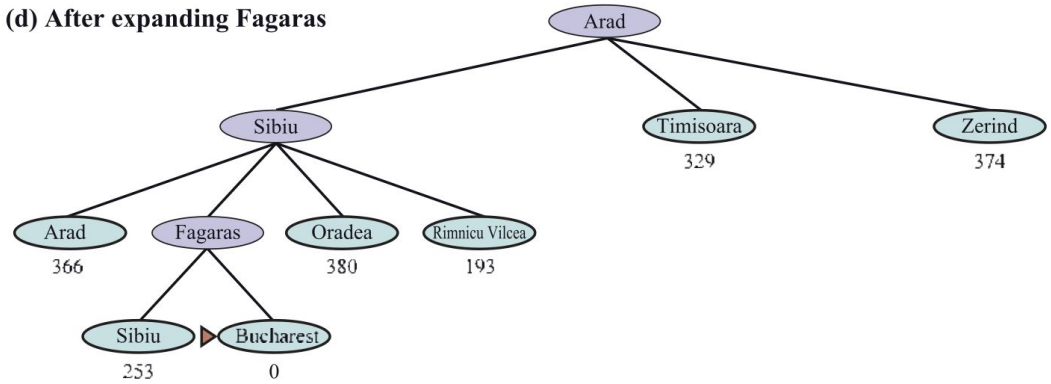
Straight-line distance can be taken as an *estimate of road distance*

Idea: expand nodes with shortest straight-line distance from Bucharest first

Greedy Best-first Search – Example



Greedy Best-first Search – Example



Greedy Best-first Search

In general:

- *h(n):* estimated cost of cheapest path form node *n* to goal node
- *h(n)* is called *heuristic function*
- *Greedy Best-first search* corresponds to Best-first search with *f(n)=h(n)*

Sound, Complete, not Optimal (as example above shows)

Time and Space complexity: $O(b^m)$

- heuristic function may greatly impact efficiency

A*-Search

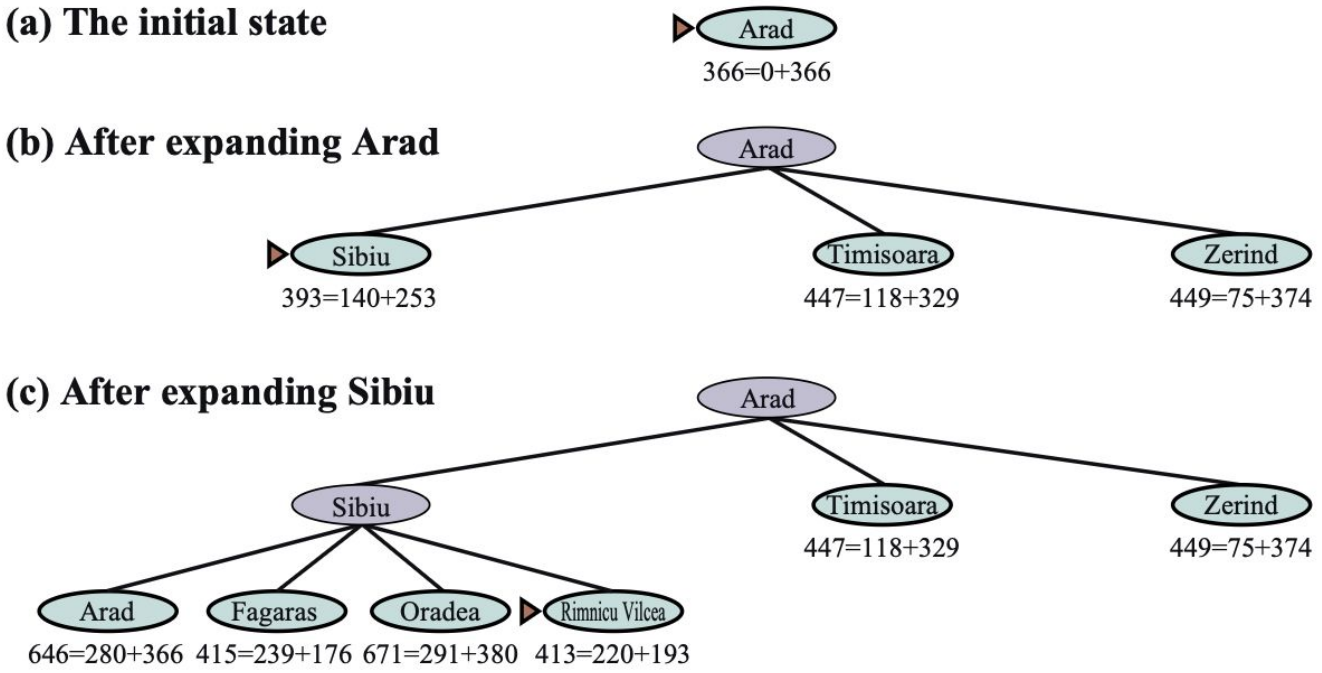
Call:

- *g(n)*: cost of path from initial state to node *n*
- *h(n)*: estimated cost of best path from *n* to some goal node

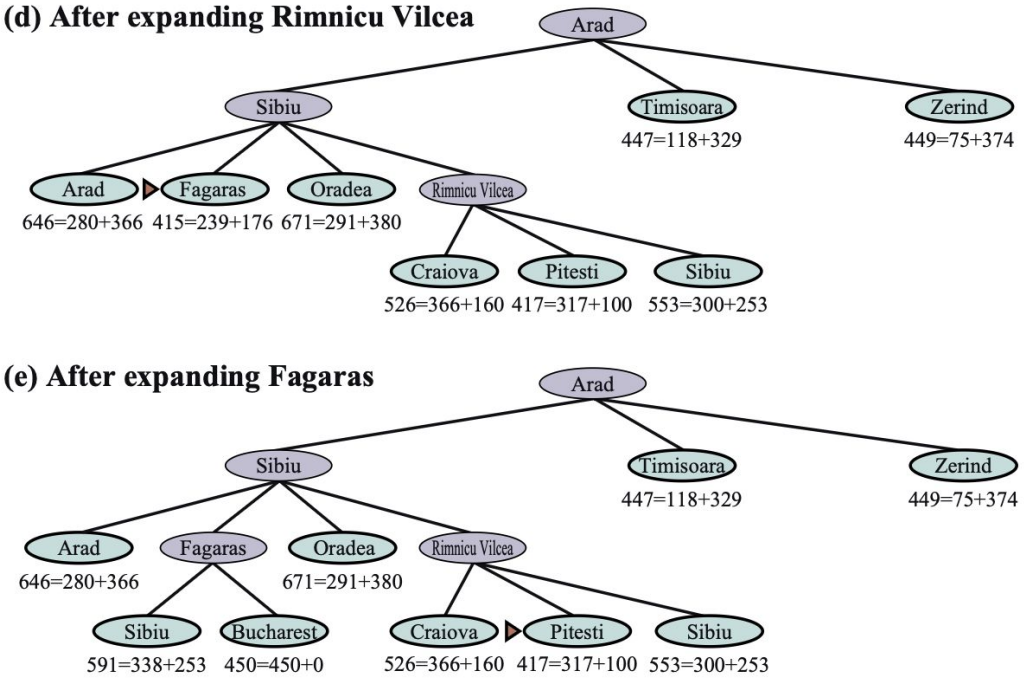
A*-Search:

- Best-first search with evaluation function *f(n)=g(n)+h(n)*
- (Intuitively, combines Uniform-cost Search with Greedy Best-first Search)

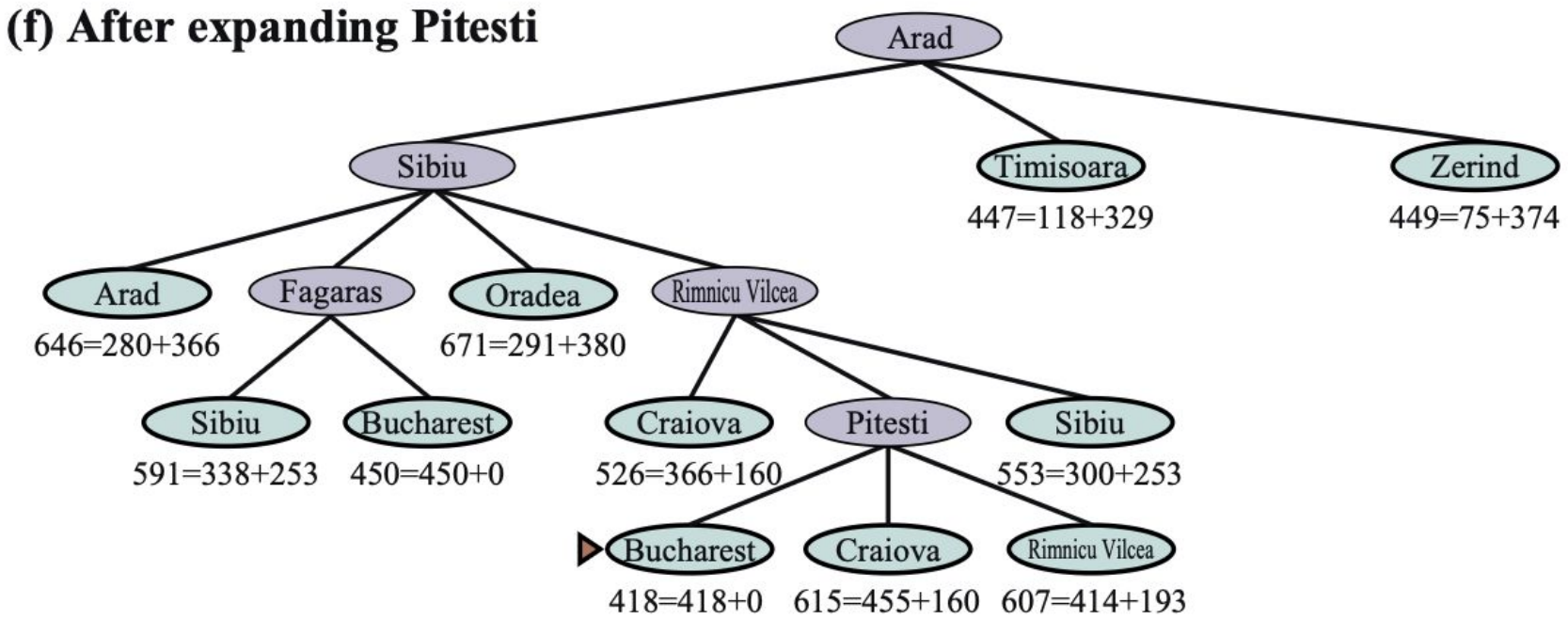
A*-Search – Example



A*-Search – Example



A*-Search – Example



A*-Search

Use of heuristic function allows for *pruning* high number of branches in ST

Properties of A*-Search:

- Sound
- Complete
- Time and Space complexity : $O(b^d)$
- Optimal? Depends on heuristic function $h(n)$

Admissible Heuristic Function

h(n) is *admissible* if it never overestimates cost of reaching goal from *n*

Theorem. If *h(n)* is admissible then A*-Search is Optimal

Observe:

- there exist pairs of problems and inadmissible h(n) for which A* is optimal

A*-Search Remarks

A* not better than Breadth-first wrt worst-case complexity

However:

- Impact of heuristic can be impressive
- Allows for *pruning* huge number of branches
- (This becomes apparent in many cases, e.g., planning)

Memory requirements can still be prohibitive for large state spaces

In practical cases optimized Depth-first approaches used, giving up optimality